LECTURES

00 Intro and syntax | 01 Data types, variables | 02 Functions | 04 Objects | ★ Cheat sheet

REVISION NOTES · JAVASCRIPT

JavaScript Core Concepts Notes

Condensed from four decks: Intro, Data types, Function and Basic Object. Code is cleaned up from the slides (curly quotes and small typos fixed). Points marked "Extra" go slightly beyond the slides.

Introduction and syntax

Deck 00 · itrofinal

What JavaScript is

- Designed to add interactivity to HTML pages.
- A scripting language, meaning a lightweight programming language.
- Usually embedded directly in HTML pages.
- **Interpreted**: scripts run without a separate compile step.
- Free to use, no licence needed.
- Object oriented: you can define your own objects and variable types.

What it can do

- Write dynamic text into a page: `document.write("<h1>" + name + "</h1>")`
- React to events (page loaded, element clicked).
- Read and change HTML elements.
- Validate form data before it reaches the server, which saves the server extra work.
- Detect the visitor's browser and load a page made for it.
- Create cookies, to store and retrieve information on the visitor's computer.

Where to put a script

```
<html>
<body>
```

```
  <script type="text/javascript">
    document.write("Hello World!");
  </script>
</body>
</html>
```

Scripts run immediately while the page loads, which is not always what you want.

PLACEMENT	USE IT FOR
`<head>`	Code that runs when called or when an event fires (functions). Loaded before anyone can use it.
`<body>`	Code that runs as the page loads and generates page content.
Both	Any number of scripts are allowed in head and body.
External `.js` file	Same code on several pages. Use `<script src="xxx.js"></script>` where you would normally write the script.

WATCH OUT

An external `.js` file must **NOT** contain the `<script>` tag. The file holds plain JavaScript only.

```
// js/ex1.js
function call(){ document.write("hello welcome to javascript"); }

<head><script src="js/ex1.js"></script></head>
<body><script> call(); </script></body>
```

Variable lifetime

- **Local**: declared inside a function. Only that function can use it, and it is destroyed when the function ends. Two functions can each have a local with the same name.
- **Global**: declared outside a function. Every function on the page can use it. It lives from its declaration until the page is closed.

Operators

ARITHMETIC	EXAMPLE
+	2+2 = 4

-	5-2 = 3
*	5*4 = 20
/	15/5 = 3, 5/2 = 2.5
% remainder	5%2 = 1, 10%8 = 2, 10%2 = 0
++	x=5; x++ gives 6
--	x=5; x-- gives 4

ASSIGNMENT	SAME AS
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y
x %= y	x = x % y

COMPARISON	MEANING	EXAMPLE
==	Equal in value	5==8 false
===	Equal in value **and type**	x=5, y="5": x==y true, x===y false
!=	Not equal	5!=8 true
> < >= <=	Greater, less, or equal variants	5>=8 false, 5<=8 true

Variables and naming

- A variable is a container for information. Its value can change while the script runs.
- Names are **case sensitive**: `strname` and `STRNAME` are different. The same goes for functions: `myfunction` is not `myFunction`.
- A name must begin with a letter or underscore.

parseInt(string, radix)

PARAMETER	MEANING
`string`	Required. The text to parse.
`radix`	Optional. Number system, from 2 to 36 (16 = hexadecimal).

- Only the **first number** in the string is returned. Leading and trailing spaces are allowed.
- If the first character cannot be converted, the result is `NaN`.
- Radix omitted: a string starting `0x` is read as hex (16); starting `0` was read as octal (8, deprecated); anything else is decimal (10).

```
parseInt("42px")       // 42
parseInt("ff", 16)     // 255
parseInt("abc")        // NaN
```

Code layout rules

- Extra white space is ignored: `name="Hege"` and `name = "Hege"` are the same.
- You can break a line **inside a text string** with a backslash. You cannot break it elsewhere, for example between `document.write` and its bracket.
- Comments: `// single line` and `/* multi-line block */`

Properties and methods

- **Properties** are values attached to an object. **Methods** are actions the object can perform.

```
var txt = "Hello World!";
document.write(txt.length);              // property: 12
var str = "Hello world!";
document.write(str.toUpperCase());       // method: HELLO WORLD!
```

Browser object model (DHTML DOM)

`window` is the top object. Under it: `history`, `document`, `location`, `screen`, `navigator`, `frames`, `event`. Under `document`: `all`, `children`, `selection`, `forms`, `body`, `links`, and the form controls (text, button, radio, textarea, select, password, file, checkbox, submit, reset, option).

Data types, variables and scope

Deck 01 · datatype

How a page runs

- Execution is **top to bottom**. Code in the head runs before code in the body.
- All JavaScript must sit inside `<script></script>`. HTML does not understand script.
- The browser has a JavaScript engine that compiles your code to native code. JavaScript can be disabled in the browser.
- `document` is a reference to the Document object and `write` is a method of it. `document` belongs to `window`. Use `console.log(document)` and `console.log(window)` to see every property and method.

Unassigned, undeclared, and the strict mode

SITUATION	RESULT
Declared, not initialised: `var a;` then print `a`	`undefined`
Never declared and read: print `a`	`ReferenceError: a is not defined`
Never declared but assigned: `a = 5;`	No error. Creates a global variable (non-strict mode)
Same assignment under `'use strict'`	`ReferenceError: assignment to undeclared variable a`

WHY THIS MATTERS

Declaring is not compulsory in JavaScript. If you write `Fname = "VITA"` and later mistype `Fnme`, there is no error: `Fnme` simply becomes a new variable. Use `'use strict'` so mistakes like that fail loudly. In strict mode you must declare with `var`, `let` or `const`.

```
'use strict'
a = 5;     // ReferenceError
```

typeof and the basic types

```
typeof 5          // "number"
typeof 5.5        // "number"   (no separate int/float)
typeof "hello"    // "string"
typeof 'h'        // "string"   (no char type)
typeof true       // "boolean"
```

```
typeof true      // "boolean"
typeof NaN       // "number"
typeof null      // "object"   (a known quirk)
var y;  y        // undefined
y + 2            // NaN   (undefined with a number)
```

The + operator is overloaded

If either side is a string, `+` joins text. Otherwise it adds. Evaluation goes left to right, and brackets go first.

```
var a = 5, b = 9;
a + b                    // 14
a + b + " ans "          // "14 ans "
"ans=" + a + b           // "ans=59"   string first, so everything after is joined
"ans=" + (a + b)         // "ans=14"   brackets run first
```

Automatic type conversion

JavaScript is **loosely typed**: a variable can hold any type and change type later (`x = "Hello"; x = 5;`).

EXPRESSION	RESULT	TYPE
`5 + 7`	12	number
`5 + "7" / "5" + 7`	"57"	string
`5 - 7`	-2	number
`5 - "7" / "5" - 7`	-2	number
`5 - "y"`	NaN	number
`"5" * "7"`	35	number

RULE OF THUMB

Only `+` prefers strings. `-` , `*` and `/` always try to turn both sides into numbers. A value's real content can be seen with `.valueOf()` .

var vs let vs const

	VAR	LET	CONST
Scope	Function. Declaration is hoisted to the	Block	Block

	top		
Redeclare in same scope	Allowed, no error	SyntaxError	SyntaxError
Reassign	Yes	Yes	No
Must initialise at declaration	No	No	**Yes**

```
for (var i = 1; i <= 3; i++) document.write(i);
document.write(i);   // 4   i is still visible after the loop

for (let i = 1; i <= 3; i++) document.write(i);
document.write(i);   // error, i exists only inside the loop block

var a = 55; var a = 99;     // fine
let a = 99; let a = 55;     // SyntaxError: redeclaration of let a
```

The slides conclude that `let` is preferred over `var`, because redeclaring by mistake is caught.

const in detail

```
const MAX_ITEMS = 30;      // valid
const NAME;                // SyntaxError: missing initialization
const NAME; NAME = "VITA"; // SyntaxError: must initialise on the same line

if (condition) { const MAX_ITEMS = 5; }
// MAX_ITEMS is not accessible here (block scope)
```

WATCH OUT

`const` makes the **VARIABLE** read-only, not the value inside it. You cannot reassign the object or array, but you can change its contents.

```
const MY_OBJECT = { key: 'value' };
MY_OBJECT = { OTHER_KEY: 'value' };   // TypeError: Assignment to constant variable
MY_OBJECT.key = 'otherValue';         // allowed
MY_OBJECT.Name = 'vita';              // allowed

const MY_ARRAY = [];
MY_ARRAY.push('A');                   // allowed
MY_ARRAY = ['B'];                     // TypeError
```

```
Object.freeze(MY_OBJECT);             // makes the object contents immutable

const arr = [1, 2, 3];
arr.push(5);
for (const item of arr) console.log(item);   // 1 2 3 5
```

Functions

Deck 02 · function

Basics

- JavaScript is dynamically typed, so a function has **no return type** and parameters have **no types**.
- A function that does not return anything gives back `undefined`.
- A function is itself a value: `typeof go` is `"function"`. Printing `go` without brackets prints its definition.
- Each call gets its own copy of the local variables.

```
function go(){ document.write("hello"); }
go();                        // call
var r = go();                // r is undefined
document.write(typeof go); // "function"

function call(a, b){
  var p = a * b;
  return p;
}
call(5, 7);   // 35
call(3, 3);   // 9
```

Too few arguments is not an error

Missing parameters become `undefined`, so `call(5)` computes `5 * undefined` which is `NaN`.

Three ways to give a parameter a default

TECHNIQUE	CODE
Check for undefined	`if (a != undefined && b != undefined) p = a*b; else if (a != undefined) p`

Logical OR	`b = b || 3;`
Default parameter	`function call(a, b = 1)`

WATCH OUT

The slide lists `0` and `undefined` as false values. With `b = b || 3`, passing `0` also becomes 3. Falsy values in full: `0`, `""`, `null`, `undefined`, `NaN`, `false`.

Variable number of arguments

The `arguments` object holds everything passed to the function. It is not a real array.

```
function findMax(){
  var i, max = arguments[0];
  for (i = 1; i < arguments.length; i++){
    if (arguments[i] > max) max = arguments[i];
  }
  return max;
}
findMax(1, 123, 500, 115, 44, 88);   // 500
findMax(2, 9);                       // works with any count
```

Rest parameter `...name` collects the remaining arguments into a real array.

```
function multiply(...myarg){          // 3 dots
  let sum = 0;
  for (let i = 0; i < myarg.length; i++) sum += myarg[i];
  return sum;
}
multiply(2, 3);            // 5
multiply(2, 3, 5, 6, 7);   // 23
```

```
function f(m, ...rest){ }   // m takes the first, rest takes the others
function g(...rest, m){ }   // SyntaxError: Rest parameter must be last formal paramet
```

REST PARAMETER	ARGUMENTS OBJECT
Holds only arguments not given a separate name	Holds all arguments passed
A real Array: `sort` , `map` , `forEach` , `pop` work directly	Not a real array; those methods do not work directly
No extras	Has extras such as `callee`

Spread operator

The opposite of rest. Rest combines many arguments into an array; spread **splits an array** into separate arguments. It can be used anywhere in the argument list, and more than once.

```
function myFunction(x, y, z){ return x + y + z; }
var args = [0, 1, 2];
myFunction(...args);        // 3

function f(v, w, x, y, z){ }
var args2 = [0, 1];
f(-1, ...args2, 2);         // v=-1, w=0, x=1, y=2, z=undefined
```

Why 'use strict' is good practice

```
var x;                  // global
function call(){
  a = 5;                // no declaration: becomes GLOBAL (leak)
  var c = 7;            // local
  x = 9;
}
call();
document.write(a);      // 5   visible outside, only after call() ran
document.write(x);      // 9
// c is out of scope here
```

Without strict mode, a forgotten `var` silently creates a global variable.

Objects

Deck 04 · basic_Object

Three ways to create an object

STYLE	CODE
new Object()	`var obj = new Object(); obj.name = "vita";`
Literal	`var obj = { name: "raj", marks: 80 };`
Constructor function	`function person(nm, mks){ this.name = nm; } var ob = new person("mona"`

Literal object

```
var obj = { name: "raj", "2division": "A" };
obj.marks = 80;                          // add a member directly
alert(obj);                              // [object Object]
document.write(obj.constructor);         // function Object
document.write(obj.name + " " + obj["2division"]);
```

- Uses a colon `:` to give values, and members are separated by **commas**.
- If a member name is not a legal variable name (starts with a digit, for example), put it in quotes and read it with `obj["2division"]`.
- Printing an object calls `toString()`. Because the object has none of its own, the parent `Object`'s version runs and prints `[object Object]`.

Constructor function as a class

Constructor function as a class

```
function person(nm, mks){
  this.name = nm;       // 'this' makes it an instance member
  this.marks = mks;
}
var ob = new person("mona", 80);

ob.name; ob["name"];                 // both read the member
for (let pp in ob) document.write(pp + " " + ob[pp]);   // name mona, marks 80
ob instanceof person;                // true
ob instanceof Object;                // true
ob.constructor;                      // prints the person function
```

- `new person(...)` sends control into the function, which initialises the members.
- `this` is required. Writing only `name = nm` would not create an instance member.
- All members are public by default.
- `for...in` loops over the member names; `ob[pp]` reads the value for each.

Literal vs constructor object

	CONSTRUCTOR	LITERAL
Members defined with	`this.x = value`	`x: value`
Separator	Semicolons allowed	Commas required, no semicolons between members
How many objects	Many	One

Forgetting `new`

Calling `person("raj", 30)` without `new` is allowed. `this` then refers to **window**, so `name` and `marks` are created on the window object. The slide calls this the biggest problem in this style.

With `"use strict"`, `this` is `undefined` in that case, so the call fails with an error instead of polluting window. The inheritance example guards with `if (this instanceof person) ... else throw new TypeError(...)`.

Copying objects

METHOD	WHAT HAPPENS

`var cpy = ob;`	Not a copy. Both names point at the **same** object. `cpy.name = "geeta"` also changes `ob.name`
`Object.assign(target, source)`	Copies own enumerable properties from source to target and returns the target (`returned === target` is true). **Shallow**
`{ ...user }` spread	New object with copied properties (`clone === user` is false). Shorter than assign. **Shallow**

```
const target = { a: 1, b: 2 };
const source = { b: 4, c: 5 };
Object.assign(target, source);        // target is { a: 1, b: 4, c: 5 }

const clone = Object.assign({}, source);   // clone with an empty target

const user = { name: 'Alex', address: '15th Park Avenue', age: 43 };
const updated = { ...user, salary: 12345 };   // add a property
const older   = { ...user, age: 56 };         // update a property
// user is unchanged in both cases
```

SHALLOW MEANS NESTED OBJECTS ARE SHARED

If a property holds another object, only the reference is copied.

```
const user = { name: 'Alex', address: { dd: 'Juhu' }, age: 43 };
const clone = { ...user };
user.address.dd = 'vita';
clone.address.dd;     // 'vita'  the nested object is the same one
```

Extra: `structuredClone(user)` makes a deep copy in modern browsers.

Adding methods

Literal object: put a function as a member. Call another member with `this.` . Defining `toString` controls how the object prints.

```
var obj = {
  fname: "Raj", lname: "Mathur", age: 25,
  add: { d: "juhu" },                       // nested literal object
  fullname: function(){ return this.fname + " " + this.lname; },
  toString: function(){ return this.fullname() + " " + this.age + " " + this.add.d; }
};
obj.fullname();      // "Raj Mathur"
document.write(obj); // uses toString: Raj Mathur 25 juhu
```

Constructor object: two places to put a method

Constructor object: two places to put a method:

```
function person(nm, ag){
  var x = "blue";                          // private: only visible inside this functio
  this.name = nm;
  this.age = ag;
  this.speak = function(){ return " hello " + this.name + " " + x; };   // instance me
}
person.prototype.walk = function(){ return this.name + " can walk "; };  // prototype
person.prototype.toString = function(){ return this.name + " " + this.age; };

var obj1 = new person("raj", 80);
obj1.speak();     // hello raj blue
obj1.walk();      // raj can walk
document.write(obj1);   // raj 80
```

INSTANCE METHOD (THIS.SPEAK = ...)	PROTOTYPE METHOD (PERSON.PROTOTYPE.WALK = ...)
A separate copy for every object. Slower and uses more memory	One copy shared by all objects. Better performance
Can use the private `var` variables of the constructor	Cannot see those `var` variables, since their scope is the constructor function
Each object has its own name, age and speak	Stored once in `person.prototype` ; every object uses that copy

call() and apply()

Inside a method, `this` is the object that owns it. `call()` and `apply()` run a method with a **different owner**, so one object can borrow another's method. The first argument becomes `this` .

```
var person = {
  fullName: function(city, country){
    return this.firstName + " " + this.lastName + "," + city + "," + country;
  }
};
var person1 = { firstName: "John", lastName: "Doe" };
var person2 = { firstName: "Mary", lastName: "Doe" };

person.fullName.call(person1, "Mumbai", "india");      // arguments listed one by one
person.fullName.apply(person1, ["Mumbai", "india"]);   // arguments as an array
```

CALL(OBJ, A, B)	APPLY(OBJ, [A, B])
Arguments passed separately	Arguments passed as one array. Otherwise identical

With `apply(this, ...)` outside strict mode, a non-object first argument makes the global object (window) the owner. In strict mode the value you pass is used as it is.

Inheritance

The slides show the older prototype syntax, which was replaced by `extends`.

```
function person(nm, ag){
  if (this instanceof person){ this.name = nm; this.age = ag; return this; }
  else throw new TypeError("Check this is not a function");
}
person.prototype.display = function(){ return "parent class method"; };
person.prototype.walk = function(){ return " Person can walk "; };
person.prototype.toString = function(){ return this.name + " " + this.age + " " + this

function child(nm, ag){
  person.call(this, nm, ag);                        // like super(nm, ag)
  this.speak = function(){ return " uh...ooooa "; };
}
child.prototype = Object.create(person.prototype);   // like extends
child.prototype.walk = function(){ return " child can crawl "; };   // override

var c = new child("rohan", 60);
c.display();   // inherited from person
c.walk();      // child's own version
document.write(c);
```

- `Object.create(person.prototype)` is the "extends" step: the child gets the parent's prototype methods.
- `person.call(this, nm, ag)` plays the role of `super(...)`: it runs the parent constructor on the child object. First parameter is the current object, the rest initialise the members.
- A method with the same name on the child's prototype overrides the parent's.
- If a method is not found, JavaScript looks in the parent, and finally in `Object`. That is where the default `toString` comes from.
- Extra: modern syntax is `class Child extends Parent { constructor(){ super(); } }`.

Practice task from the deck

Create a literal object with members `Rollno`, `Name`, `Mks1`, `Mks2`, a `Total` function, and a `toString` that displays the name and total marks.

```
var s = {
  Rollno: 1, Name: "Mona", Mks1: 70, Mks2: 80,
  Total: function(){ return this.Mks1 + this.Mks2; },
  toString: function(){ return this.Name + " " + this.Total(); }
};
document.write(s);   // Mona 150
```

Last-minute cheat sheet

All four decks

Likely exam questions

- What is JavaScript; interpreted vs compiled; what it can do.
- Where scripts can be placed; rules for an external `.js` file.
- `==` vs `===`; `var` vs `let` vs `const`.
- What happens with an undeclared variable (read vs assign), and what `'use strict'` changes.
- Output of `5 + "7"`, `"5" - 7`, `"ans=" + a + b`, `typeof null`.
- Default parameter techniques; rest vs `arguments`; rest vs spread.
- Ways to create an object; literal vs constructor; what `this` does.
- Shallow copy: `=` vs `Object.assign` vs spread, and the nested-object problem.
- Instance method vs prototype method.
- `call` vs `apply`; how inheritance is done with `Object.create` and `call`.

Easy mistakes

- Treating `const` as deep-frozen. Only reassignment is blocked.
- Assigning an object with `=` and expecting a copy.
- Forgetting `this.` in a constructor, or `new` when calling it.
- Putting the rest parameter or the default parameter anywhere but last.
- Using `b = b || 3` when `0` is a legal value.
- Case mistakes: `myFunction` vs `myfunction`.
- Expecting a prototype method to read a constructor's private `var`.
- Putting a `<script>` tag inside a `.js` file.
- Using a semicolon between members of a literal object. Use commas.